Hunter Worssam
October 24th, 2025

Comparative Analysis of Particle Swarm Optimization (PSO), Genetic Algorithms (GA), and Evolutionary Strategies (ES) in Optimization Tasks

Background

Evolutionary algorithms (EAs) are population-based optimization methods inspired by natural processes like evolution, adaptation, and collective behavior. Unlike gradient-based techniques, they do not rely on differentiability or continuity of the objective function, making them powerful for black-box, non-convex optimization problems.

Belowl briefly outline three EA algorithms, Particle Swarm Optimization (PSO), Genetic Algorithms (GA), and Evolutionary Strategies (ES) focusing on their architecture and benefits. Later during the implementation I'll touch more on their hyper parameters and performance.

Particle Swarm Optimization (PSO)

Architecture & Mechanism:

- PSO models a group of particles that move through the search space, where each particle can be thought of as a potential solution to the optimization problem.
- Each particle has:
  - A position vector $x_i$representing a possible solution
  - A velocity vector $v_i$controlling the particles movement
  - A record of its personal best p_i and the global best g found by the swarm algorithm
- At each iteration, the particles update their velocity and position using a function comprised of a momentum term, a nearest neighbor term, and a randomness contribution.

Where PSO Shines:

- Continuous, real-valued optimization (such as parameter tuning or control problems).
- Low-to-moderate dimensional spaces (10-100 features).
- When fast convergence is desirable.

Genetic Algorithms (GA)

Architecture & Mechanism:

- GA mimics natural selection and genetic recombination in its approach.
- It operates on a population of chromosomes, each encoding a potential solution.
- It iteratively applies:
  - Selection: Chooses the fittest individuals for reproduction (rank-based or randomly).

- Crossover (recombination): Exchange genetic material between pairs of parents to create offspring.
- Mutation: Randomly alter bits/genes to maintain diversity.
- Replacement: Form a new generation, often using elitism (retaining top individuals).

Where GA Shines:

- Combinatorial or discrete optimization (such as feature selection, scheduling, routing problems)
- I've personally seen GA's used in the development of novel gene therapy virus capsids, where GA is used to derive a library of different DNA plasmids which can then be tested in-vitro.
- Search spaces with complex constraints or non-differentiable objectives.
- When the solution encoding naturally maps to a string/bit representation.

Evolutionary Strategies (ES)

Architecture & Mechanism:

- ES evolved in Europe as a variant of GA specialized for continuous optimization.
- It uses real-valued vectors for individuals and focuses on mutation and self-adaptation rather than crossover.
- Each individual has:
  - A solution vector x
  - Strategy parameters (mutation step sizes) σ
- Mutation is Gaussian: $x' = x + \sigma \cdot N(0, I)$
- Where step sizes evolve over the iteration window according to:
  - $\sigma' = \sigma \cdot e^{\tau N(0, 1)}$

Where ES Shines:

- Continuous optimization (such as real-valued functions and hyper parameter tuning).
- Problems requiring wide global search with adaptive step control.
- High-dimensional landscapes (such as neural network weight optimization or policy search in reinforcement learning).
- Derivative-free optimization of smooth functions.

Datasets Used:

The three optimization algorithms above were tested on two different optimization use cases. In the first use case, we are interested in performing feature selection for the breast cancer dataset, which includes 30 unique features and a binary target of malignant/benign. The selected features are used in a logistic regression algorithm with a penalty term to apply selective pressure towards a smaller number of features. We will compare the number of selected features chosen by each of the three optimization algorithms, as well as the resulting accuracy of the logistic regression.

In the second use case, we leverage the three optimization algorithms to perform hyper parameter tuning for a simple multi-layer perceptron neural network on the digits dataset. The four hyper parameters that must be tuned are the learning rate, regularization strength, number of hidden units and number of layers, with the optimal solution vector being the one that minimizes validation loss.

Together, these tasks showcase how evolutionary methods can systematically optimize high-dimensional, nonlinear, and partly discrete search spaces, covering both feature subset selection and continuous model hyper parameter tuning.

Feature selection: `sklearn.datasets.load_breast_cancer()`
- (binary classification, 30 features)

Hyperparameter tuning: `sklearn.datasets.load_digits()`
- (multiclass 10 classes; 64 dimensions) tuning a small MLP

Models Used:

Three population-based evolutionary algorithms were implemented and compared: Particle Swarm Optimization (PSO), Genetic Algorithm (GA), and Evolutionary Strategy (ES). Each algorithm was designed to minimize a specific objective function in continuous search spaces under bounded constraints. Each of these algorithms were derived from first principles, not using libraries.

- PSO models optimization as the collective motion of particles influenced by their personal and global best experiences, with parameters controlling inertia and cognitive/social attraction.
- GA simulates natural selection through iterative population evolution via tournament selection, crossover (recombination), and Gaussian mutation, maintaining diversity through random variation and elitism.
- ES evolves both candidate solutions and their mutation step sizes simultaneously, allowing the algorithm to self-regulate exploration intensity via log-normal adaptation of σ parameters.

Questions to Answer:

1. Which algorithm converges quickest (fewest iterations-to-best) on each task?
2. Which yields the best solution quality (lowest loss / highest validation accuracy)?
3. How robust are results across two random seeds (standard deviation of loss)?
4. What are the computational demands (number of function evaluations, runtime)?
5. Where do exploration–exploitation trade-offs appear?

Experimental Design:

Two independent seeds (0, 1) were used per algorithm to estimate variability and robustness.

Population and iteration setpoints were kept intentionally small for fast comparative runs, though each optimizer supports scaling for more exhaustive searches.

Task	Population / Particles	Iterations / Generations	Notes
Feature Selection	10–14	8	Features are either selected [1] or discarded [0]
Hyperparameter Tuning	10–14	8	Continuous search over MLP regularization, learning rate, hidden units, and number of layers

For each run, the following metrics were recorded:

- **Best loss value** (reported as accuracy proxy = 1 – loss)
- **Iteration of best** (proxy for convergence speed)
- **Number of objective evaluations** (proxy for computational effort)
- **Wall-clock runtime (s)** (total CPU time per run)

Results

Across both optimization tasks, all three algorithms achieved strong performance, but their behavior reflected distinct strengths.

For Feature Selection, Evolutionary Strategy achieved the highest mean accuracy (97.5%), followed closely by Genetic Algorithm (97.2%) and Particle Swarm Optimization (96.9%). ES and GA required modest iterations (~4-6) and had consistent performance across seeds, while PSO converged extremely fast (2 iterations) but was not able to find the global optimum solution.

For Hyper-parameter Tuning, ES again delivered the best accuracy (98.1%), with GA (97.8%) and PSO (97.9%) close behind. PSO remained the fastest in runtime, but ES achieved the most stable and accurate results overall despite longer compute times.

Overall, ES consistently provided the best trade-off between solution quality and reliability, GA was more stable (lower standard deviation in accuracy between runs) but slower, and PSO prioritized speed and early convergence at a small cost in accuracy.

Table 1. Feature Ranking Case Study Results:

[29]:

	Task	Algorithm	Mean_Best_Loss	Std_Best_Loss	Mean_Iter_of_Best	Mean_Evaluations	Mean_Runtime_s	Std_Runtime_s	Runs	Accuracy_proxy
0	Feature Selection (Breast Cancer)	ES	0.025135	0.003192	4.5	134.0	0.105339	0.000750	2	0.974865
1	Feature Selection (Breast Cancer)	GA	0.028058	0.000471	6.0	126.0	0.097926	0.003876	2	0.971942
2	Feature Selection (Breast Cancer)	PSO	0.030649	0.002721	2.0	90.0	0.082524	0.022550	2	0.969351

Figure 1. Feature Ranking Case Study Convergence Plot

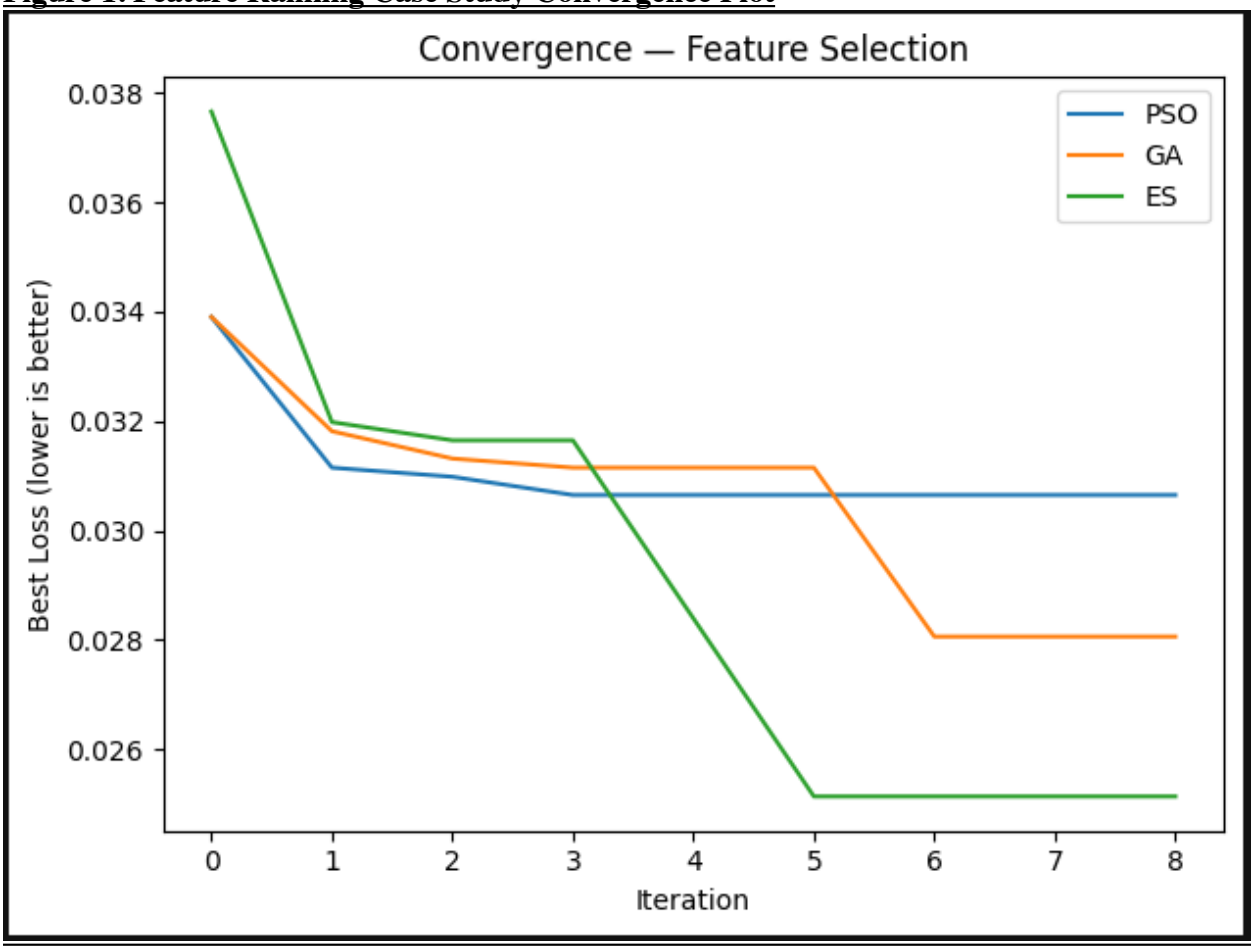
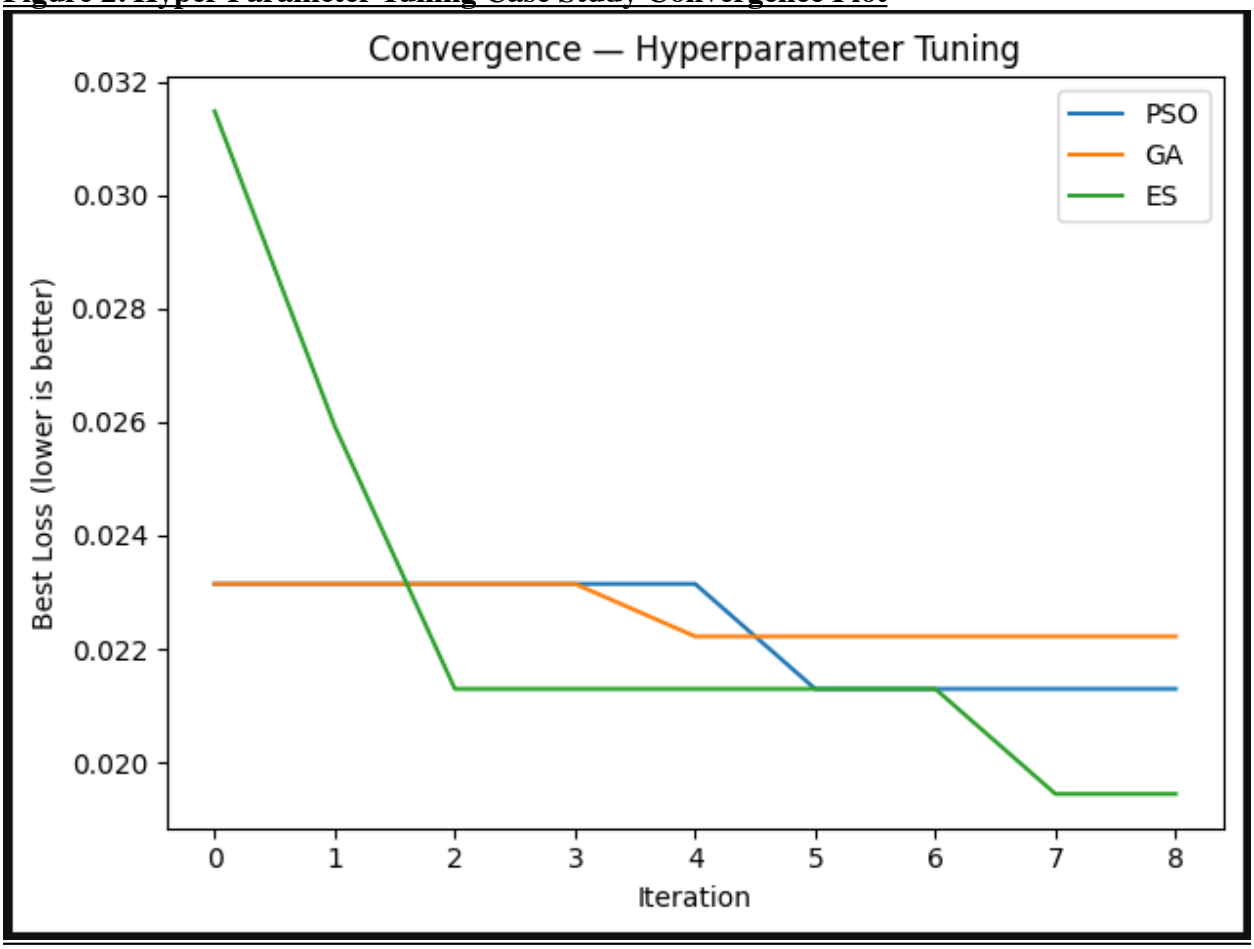


Table 2. Hyper Parameter Tuning Case Study Results:

[36]:

	Task	Algorithm	Mean_Best_Loss	Std_Best_Loss	Mean_Iter_of_Best	Mean_Evaluations	Mean_Runtime_s	Std_Runtime_s	Runs	Accuracy_proxy
0	MLP Hyperparameter Tuning (Digits)	ES	0.019444	0.001309	3.5	134.0	25.354269	5.287930	2	0.980556
1	MLP Hyperparameter Tuning (Digits)	GA	0.022222	0.002619	2.0	126.0	17.792302	0.759217	2	0.977778
2	MLP Hyperparameter Tuning (Digits)	PSO	0.021296	0.001309	2.5	90.0	9.671057	0.765109	2	0.978704

Figure 2. Hyper Parameter Tuning Case Study Convergence Plot



Discussion

Overall I was pleasantly surprised at how efficient all three of these algorithms were across both case studies. The real trick with all of these approaches is striking a balance between allowing your algorithm to explore the feature space entirely while ensuring the model is able to hone in quickly on optimal candidate solutions.

- Particle Swarm Optimization's collective learning quickly exploited promising regions but lacked sustained diversity, leading to early stagnation once the swarm clustered around local minima.
- The Genetic Algorithm maintained better diversity through crossover and mutation but required additional generations to converge, trading runtime for stability.
- The Evolutionary Strategy's self-adaptive mutation strategy offered a dynamic balance of large exploratory steps early on and finer exploitation later which allowed it to consistently achieve superior accuracy.

In computational terms, all methods were efficient: runtimes stayed below 30s even for multi-dimensional searches. The ES's longer runtime was justified by its improved

accuracy, suggesting it scales well for complex, continuous optimization where model accuracy is paramount. The robustness across seeds further indicates that all three algorithms are reliable optimizers even under limited evaluation budgets. I'd feel comfortable using any of these approaches for optimization problems, although the structure of the evolutionary approach was the must intuitive and appealing to me.

Best Practices

- **Use ES** for continuous, smooth optimization tasks where superior accuracy is the highest priority.
- **Use GA** when population diversity and robustness across runs are priorities, or when solution stability is critical.
- **Use PSO** for rapid prototyping or when computation time is constrained, acknowledging the potential for premature convergence.
- For fairness in comparisons, always fix random seeds, maintain equal population/iteration budgets, and log both loss and runtime.
- When scaling experiments, increase population size or generations gradually and monitor convergence plots to detect over-exploitation.
- **In a future study, it would be interesting to see how the algorithm performance changes as population size, iteration size and step change size are scaled up or down.**

References

Kennedy, J., & Eberhart, R. (1995). *Particle swarm optimization.* Proceedings of IEEE International Conference on Neural Networks.

Holland, J. H. (1975). Adaptation in Natural and Artificial Systems. University of Michigan Press.

Rechenberg, I. (1973). Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution. Frommann-Holzboog.

Bäck, T., Fogel, D. B., & Michalewicz, Z. (1997). *Handbook of Evolutionary Computation.* Oxford University Press.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Particle Swarm Optimization (PSO):
Clerc, M., & Kennedy, J. (2002). The particle swarm—Explosion, stability, and convergence in a multidimensional complex space. IEEE Transactions on Evolutionary Computation, 6(1), 58–73.

Genetic Algorithms (GA):
Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley.

Evolution Strategies (ES):
Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. Evolutionary Computation, 9(2), 159–195.